DUAL AGENT / PRACTICAL GUIDE

图解增强版

聪明 Agent 指挥执行 Agent

你只提需求，主 Agent 负责派工

你只说分工、方案意见、批准和反馈。执行 Prompt 与子智能体提示词，由主 Agent 自动编写。

Kimi K3.1 Pro、Codex、Claude 可作为指挥层，DeepSeek、GLM/ZCode、Codex 等可作为执行层。按实际对话方式整理，示例均为虚构。

01 / 指挥层

规划与审查

需求 · 派工 · 证据 · 质量

02 / 执行层

实施与测试

修改 · 日志 · 产物 · 交接

03 / 独立复核

PASS 或返工

只读 · FAIL · 未验证

作者：simon，一个不懂代码的AI极客

微信交流群：simonsxxs

GitHub：https://github.com/simonsxxs　Gitee：https://gitee.com/simonsxx　个人网站：https://simonsxx.com

V3.3 · 2026-10-25

自然语言版 | 不用写执行 Prompt | 不用记 CLI 命令

CONTENTS / 阅读导航

你提需求，主 Agent 处理其余工作

00—01 分清分工 · 02—04 方案、批准与派工 · 05—07 进度、验收与返工 · 08—09 全局规则与模型组合 · 10 常用话术 · 附录 A—D 自动生成模板

分工：00—01 | 方案与批准：02—04 | 执行与验收：05—07 | 内部模板：附录 A—D

00 先说清：你不需要写执行 Prompt 3

01 第一步：告诉主 Agent 怎么分工 4

02 第二步：先讨论方案，不急着执行 5

03 第三步：方案确认与实施批准，是两件事 6

04 第四步：一句批准，主 Agent 开始派工 7

05 执行期间：你可以继续做其他事 8

06 结果回来：你只问这三个问题 9

07 不满意：直接说哪里不对 10

08 想更省事：让主 Agent 写全局规则 11

09 换成其他模型，说法不变 12

10 一页速查：你常用的几句话 13

附录 A：主 Agent 自动生成的执行 Prompt 14

附录 B：主 Agent 自动生成的独立审查 Prompt 15

附录 C：主 Agent 自动生成的定向返工 Prompt 16

附录 D：主 Agent 自动整理的全局 AGENTS.md 示例 17

11 作者与资料说明 18

主教程只需要自然语言。常用话术看第 10 章；完整执行、审查、返工和全局规则见附录 A—D，均由主 Agent 自动生成并发起。

00 先说清：你不需要写执行 Prompt

这本教程只讲一件事：**你怎样用普通中文，让主 Agent 替你组织另一个 Agent 工作。**

你不需要学习执行 Prompt 的格式，也不需要自己写子智能体提示词、任务书、测试命令或结果 JSON。那些都是主 Agent 接到你的要求后，自己生成和管理的材料。

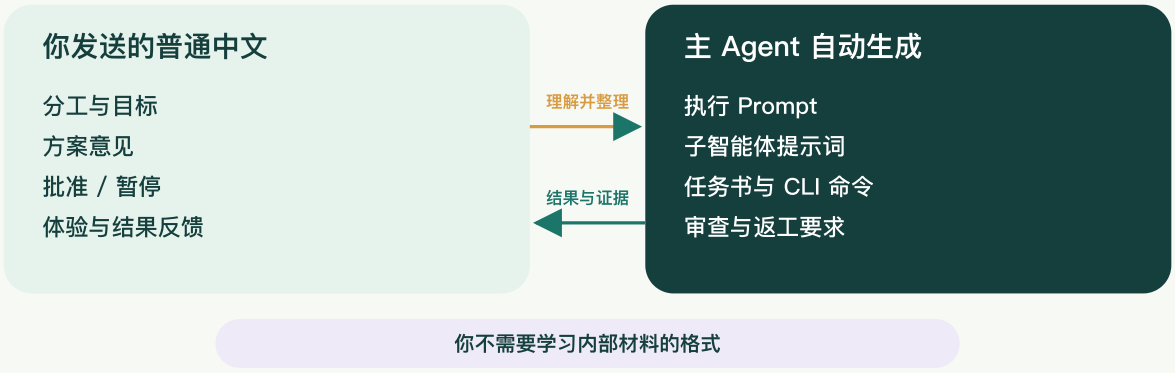


图 1 · 你与主 Agent 分别写什么 你写自然语言决定，主 Agent 写详细工作指令。

你写的和主 Agent 写的，不是一回事

你写的：普通中文

- “你出方案，让 ZCode 执行，最后你审查。”
- “可以按照你的方案。”
- “批准执行：……”
- “这里还不对，修好后再验证。”

主 Agent 自动写的：内部工作材料

- 执行 Prompt、子智能体提示词。
- 任务书、CLI 命令、测试与证据要求。
- 返工任务、独立审查要求、结果汇总。

你负责做决定，主 Agent 负责把决定变成详细工作指令。附录 A—D 展示主 Agent 自动生成的完整执行、审查、返工和全局规则模板，仅供理解，不要求你编写。

01 第一步：告诉主 Agent 怎么分工

第一次使用时，你只需把“谁负责方案、谁负责执行、谁负责审查”说清楚。

你可以直接这样说

你发送：

接下来的任务由你负责需求分析、方案设计和最终审查，具体实施交给 ZCode CLI。 请根据任务依赖安排串行或并行，自行编写执行 Prompt、子智能体提示词和任务书。 每批完成后核验实际结果；未经我确认，不进行付费操作、部署、发布或外部写入。

主 Agent 接到这句话后，应该记住的是工作方式，而不是立刻修改代码：

- 它先了解你的需求和项目。
- 它负责方案、任务拆分和质量检查。
- 它把具体执行交给独立的 ZCode 会话。
- 它自己编写执行 Prompt 和子智能体提示词。



图 2 · 一次任务怎样推进 方案讨论与实施批准分开，主 Agent 不应跳过你的确认。

如果你已经有明确任务，可以一起说

我想优化这个页面的按钮。请先只读检查各按钮的作用和重复入口，给出保留、合并与悬浮窗联动方案。 你负责方案和最终审查，具体实施交给 ZCode；我确认方案前不要修改代码。

02 第二步：先讨论方案，不急着执行

你可以先只说想法，不需要一上来讲清完整需求。

你提问题，主 Agent 做调研

你说：“这块怎么设计比较好？你看看市面上有没有可参考的做法。”

主 Agent 应该做：查看现状、研究可选方案、说明推荐方向，以及哪些事情暂不处理。

方案讨论阶段，你可以继续补充：

- “这些按钮感觉可以放到同一排，你觉得呢？”
- “手机端也要能用，别只考虑电脑。”
- “先不要改代码，出了方案告诉我。”
- “真实支付、发布和部署这次都不做。”

方案出来后，你主要检查四件事

做什么	不做什么	怎么做	怎么验收
是不是你真正想要的结果？	有没有擅自扩大范围？	页面、流程和异常处理是否合理？	最后看什么页面、操作或结果？

方案不满意就直接说哪里不对。不要让主 Agent 带着你并不认可的方案进入实施。

03 第三步：方案确认与实施批准，是两件事

这是你实际交互里最重要的一点：**“可以按照你的做”不一定等于“现在开始改代码”。**

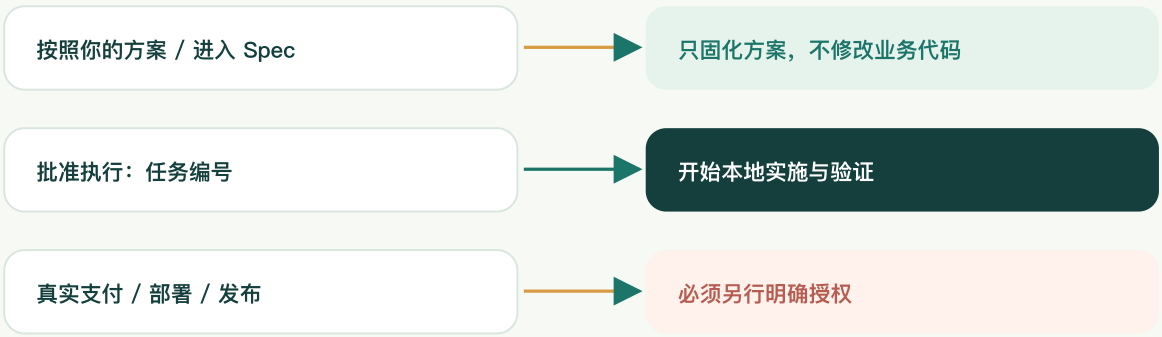


图 3 · 方案确认与实施批准　“可以按照你的做”与“现在允许实施”要表达清楚，真实操作还要单独确认。

你只确认方向时

方案方向可以。请先进入 Spec，补全功能规则和验收标准；这一步不要修改业务代码。

`Spec` 就是把方案、规则、异常和验收标准写成正式文档。主 Agent 应该继续完善方案，而不是马上派执行 Agent 改代码。

你确认可以实施时

批准执行：demo-feature-v1。请按已确认的方案范围实施。

`demo-feature-v1` 是虚构的任务编号。实际使用时，主 Agent 会在方案里给出编号，你只需按方案确认，不需要自己编一个。

如果你希望方案和批准一次完成

你也可以明确说：“进入 Spec，并在方案固化后直接批准执行：。实施范围以方案文档为准。”这句话表达的是完整授权，主 Agent 不应再把它理解成只讨论方案。

04 第四步：一句批准，主 Agent 开始派工

你批准之后，不需要再写一份任务书给 ZCode，也不需要打开终端输入命令。主 Agent 会开始处理以下工作。



图 4 · 主 Agent 自动完成的工作 你看到内部提示词或任务书，不代表这些内容应该由你提前编写。

你在界面上可能看到这些内容

- 主 Agent 更新待办，先检查当前状态和已有测试。
- 主 Agent 创建任务书，然后后台启动 ZCode。
- ZCode 返回执行日志、修改文件和测试结果。
- 主 Agent 阅读结果、查看实际产物，再决定通过还是返工。

这些内容是**主 Agent 的工作过程，不是要求你补写的材料**。你只需要在权限确认、关键方案变化或真实操作前作决定。

你可以问进度，但不用盯着每一步

> 请汇报当前进度：已完成哪些批次、正在执行什么、是否遇到阻塞，以及哪些内容尚未验证。若有权限或文件冲突，先暂停并告诉我。

主 Agent 应该如实回答“正在执行、已完成哪部分、还有什么没验证”，而不是只说“一切顺利”。

05 执行期间：你可以继续做其他事

后台执行的意义是：你不必一直等在当前对话里，也不必让主 Agent 逐行把执行日志念给你。

你可以继续做

- 讨论另一个功能或另一个方案。
- 阅读主 Agent 提交的阶段性结果。
- 处理不依赖本次执行结果的其他工作。
- 让另一个会话做**不冲突**的任务。

> 让这个会话继续执行。我们可以讨论另一部分，但不要修改它正在处理的文件；如果任务范围有重叠，先串行处理。

你不应该自己做

- 在同一批文件里同时手动修改，覆盖执行 Agent 的工作。
- 看到长时间运行就反复催它重启。
- 把另一个会话的截图、测试或结果直接当成本任务的验收证据。

如果两个任务会改同一个页面、同一份样式或同一个测试环境，先让主 Agent 判断顺序。**不确定是否冲突时，串行处理更简单。**

06 结果回来：你只问这三个问题

执行结束后，你不需要先读 JSON，也不需要懂每一条测试命令。先问下面三个问题。

01

我要的功能真的做好了吗?

请主 Agent 对照你的需求逐项说明，而不是只列修改了哪些文件。

02

你实际看过、点过、测过吗?

需要页面就看页面，需要按钮就实际点击，需要下载就打开下载产物。

03

还有什么没验证?

让主 Agent 明确列出失败、未验证和未做事项，不把局部完成说成全部完成。

你可以直接这样要求验收

> 请不要只回复“已完成”。请展示实际页面或产物，逐项说明验证了哪些操作及结果，并列出剩余 FAIL、未验证和未做事项。

主 Agent 应该把“进程结束、测试通过、实际场景通过、需求验收通过”分开说明。听到“全部完成”时，你仍然有权利查看证据。

07 不满意：直接说哪里不对

你不需要替主 Agent 写返工 Prompt。只要指出你看到的问题，以及不希望它扩大修改范围。

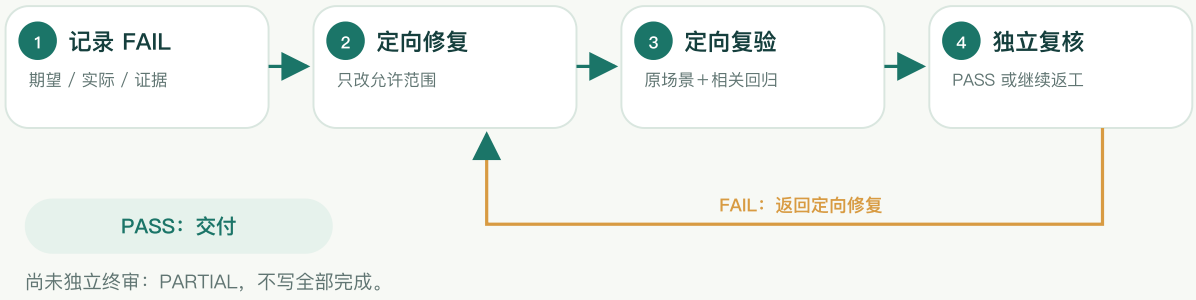


图 5 · 不满意后的返工闭环 你只指出问题和期望，主 Agent 负责整理返工任务并复验。

你可以这样说

> 手机端这个按钮仍然遮挡正文。请只修复这个问题，不改变其他设计；修复后重新验证手机与桌面端，并检查相关功能是否出现回归。

主 Agent 应该把这句话整理成内部返工任务：问题是什么、期望是什么、证据在哪里、允许改什么、修完怎样复验。

如果主 Agent 说已经修好

继续问：

- “这个问题实际重新点过了吗？”
- “原来的其他功能有没有受影响？”
- “独立审查做了吗，还是执行者自己检查自己？”

你负责指出体验和目标是否正确，主 Agent 负责技术定位、返工和验证。

08 想更省事：让主 Agent 写全局规则

如果你经常采用同一种分工，不必每次重复“你出方案、ZCode 执行、你审查”。你可以让主 Agent 把这套工作方式写进全局 `AGENTS.md`。

`AGENTS.md` 是供 Agent 阅读的规则文件。**它的内容由主 Agent 整理，你负责审核和确认，不需要自己学习文件格式。**

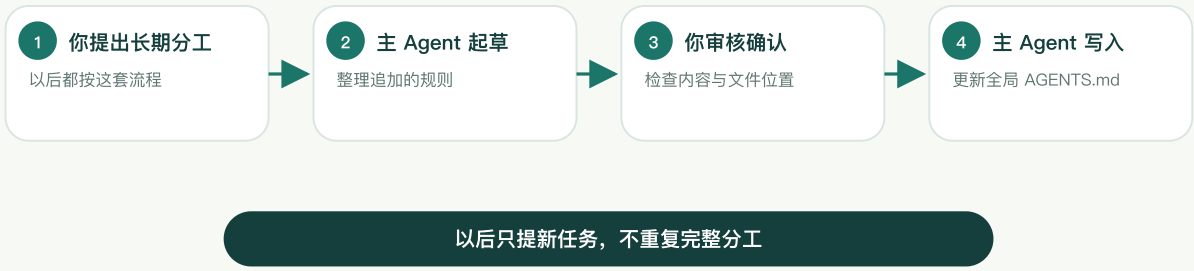


图 6 · 全局规则由主 Agent 整理　你审核规则并确认写入，不需要自己学习 AGENTS.md 的写法。

你可以这样要求

以后这类任务默认采用上述分工：你负责方案和最终审查，ZCode 负责执行。 请先把拟追加到全局 AGENTS.md 的内容、文件位置和生效方式给我审核；我确认后再写入，不覆盖已有规则。

主 Agent 应该先给出准备追加的内容、文件位置和生效方式。你确认后，它才修改文件。

以后你只需要说任务

按我们约定的分工处理这个功能。请先给出方案，我确认后再实施。

如果主 Agent 没有按约定分工，直接提醒：“按全局规则，执行交给 ZCode，你负责审查。”规则是否生效，要看它是否实际这样行动，而不是听它说“已经记住”。

09 换成其他模型，说法不变

这套交互方式不依赖“Kimi＋ZCode”这一对组合。你只需要替换角色名称，自己的说话方式不变。

组合时还可以同时考虑能力和成本：让能力较强、成本较高的模型负责方案、审美审查和质量审查；让 DeepSeek 等成本较低的模型承担规则明确、重复耗时的执行任务。

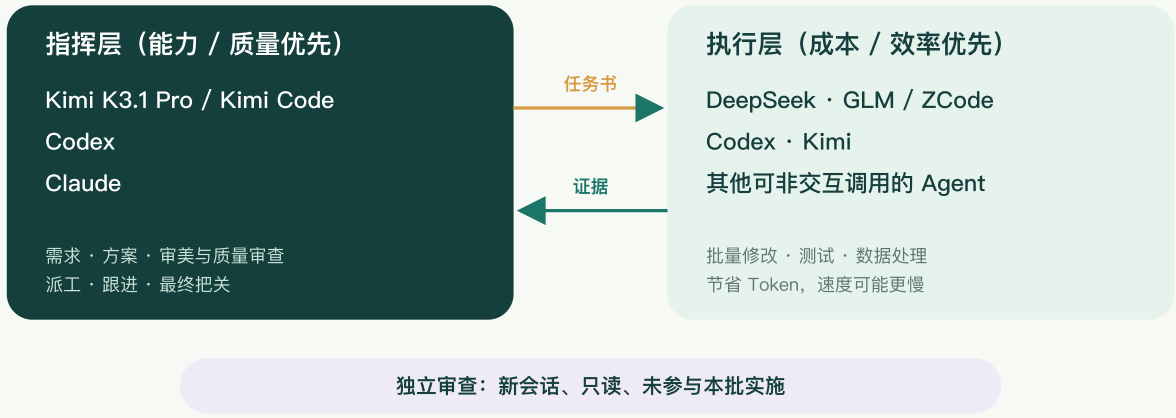


图 7 · 按能力与成本分工 DeepSeek 等低成本模型可承担规则明确的执行，节省 Token 但可能更慢。

你可以这样选择

- **指挥层：Kimi K3.1 Pro＋Kimi Code、Codex 或 Claude**：负责需求分析、方案、审美审查、质量审查与最终把关。
- **执行层：DeepSeek、GLM/ZCode、Codex 等**：负责规则明确的代码修改、批量处理、测试、日志和证据整理。DeepSeek 可以作为控制成本时的执行候选。
- **独立审查：Claude 或其他新会话**：增加一次未参与实施的复核。

例如：

> 这次由 Kimi 负责方案、审美审查和质量审查，DeepSeek 负责批量修改、测试与资料整理。请自行生成执行与审查提示词；未经我确认，不要进行真实付费操作或发布。

这种分工的收益是让模型优势互补：指挥层减少错误和返工，执行层节省 Token、降低调用费用；常见代价是执行可能更慢，批量任务和多轮返工会拉长总耗时。

模型能力和价格会随版本、平台与任务变化。**低成本不等于低质量，高成本也不等于免检。**先用小任务验证组合，同时比较质量、总耗时和费用，再决定谁指挥、谁执行；无论由谁执行，主 Agent 都必须核验实际结果。

10 一页速查：你常用的几句话

分工

你负责需求分析、方案和最终审查，具体实施交给 ZCode CLI。请自行整理内部提示词和任务书，每批完成后核验实际结果。

先出方案

请先只读调研并给出推荐方案，说明做什么、不做什么和怎样验收；我确认前不要修改代码。

确认实施

批准执行：。请按已确认的方案范围实施。

查看进度

请说明已完成批次、当前进度、阻塞和未验证项；如有权限或文件冲突，先暂停并告诉我。

要求验收

请展示实际页面或产物，逐项说明验证结果，并列出剩余 FAIL、未验证和未做事项。

定向返工

请只修复我指出的问题，不改变其他设计；修复后重新验证并检查相关回归。

暂停

请暂停当前执行和后续批次，保留已有结果，不再修改，等我确认后继续。

写入全局规则

请把这套分工整理成全局 AGENTS.md 的追加内容，并说明文件位置；先给我审核，我确认后再写入，不覆盖已有规则。

> **你不需要把话说得很技术。说清楚目标、限制、确认和反馈，剩下的详细提示词交给主 Agent。**

附录 A：主 Agent 自动生成的执行 Prompt

自动生成说明：以下模板由主 Agent 根据用户目标、项目规则和批准范围自动整理并发起，不需要用户编写，也不需要用户手动转发给执行 Agent。占位符由主 Agent 按实际情况替换。

```
你是 <change-id> 第 <N> 批执行 Agent。请实际实施和验证，不只返回建议或计划。
主 Agent 负责需求、边界与独立审查；不要替主 Agent 宣布最终验收。

一、项目与必读输入
- 唯一项目根：<项目绝对路径>。
- 先读：项目 AGENTS.md、已批准方案、当前任务和基线。
- 历史结果只能作为待复核线索，不能直接当作本次通过结论。

二、授权与前置条件
- 已获授权：<准确的本地实施范围>。
- 开始前必须满足：<稳定基线、上游接口、测试环境等>。
- 前置条件不满足时停止并报告，不提前实施。

三、本批唯一目标
<用一句话说明本批只解决什么问题，不扩大范围。>

四、允许与禁止
- 只允许修改：<明确文件、目录和证据输出位置>。
- 禁止修改：<其他会话正在使用的文件和本批范围外文件>。
- 未经用户另行明确授权，不安装依赖、不提交或推送、不部署发布、
  不真实支付、不执行外部平台写入。
- 不读取或输出 .env、密钥、Token、Cookie 和其他敏感信息。
- 不使用 reset、checkout、整文件覆盖或删除证据来处理其他会话的改动。

五、实施要求
1. <可检查的功能、页面、数据处理或文档交付项>。
2. <必须保持不变的旧行为和相关回归要求>。
3. <需要同步的接口、合同、配置或文档>。
4. <空输入、长输入、部分失败、未知结果等异常处理>。

六、验证与证据
- 运行项目现有的相关测试、完整测试、类型检查和构建，按实际任务选择。
- 实际走通用户端到端场景，不能只用文件存在、导入成功或编译通过代替。
- 保存实际命令、退出码、日志、截图、下载产物和关键测量结果。
- 核对证据来自同一版本、同一输入和正确工作区。
- mock 只证明本地模拟链路，不冒充真实模型、支付、发布或生产结果。

七、停止条件
- 必须越权、安装依赖、改变核心合同、迁移数据或删除重要内容时停止。
- 发现基线已变化、任务要求冲突或同一问题反复失败时停止并报告。
- 不通过删测试、降低阈值、缩小场景、强制点击或覆盖证据来制造 PASS。

八、交接
- 列出修改文件、每项修改原因、实际命令与结果、证据路径。
- 列出已通过项、剩余 FAIL、未验证、未做项和执行 sessionId。
- 只勾选真实完成且验证通过的任务，不归档、不发布、不替用户验收。
```

附录 B：主 Agent 自动生成的独立审查 Prompt

自动生成说明：独立审查任务由主 Agent 在收到执行结果后自动发起。审查者必须是未参与本批实施的独立会话，用户不需要为它编写审查提示词。

```
你是未参与本批实施的独立审查 Agent。只读审查，不修改代码、测试、配置、
文档或证据，不安装依赖，不提交、部署、发布或执行真实外部写入。

一、审查输入
- 已批准方案与验收标准：<路径>
- 当前任务书与基线：<路径>
- 执行结果、修改文件、原始日志和真实产物：<路径>
- 历史材料只能作为线索，必须核对当前版本和证据身份。

二、独立复核要求
1. 逐项对照需求，不采信执行者总结、测试数量或“内部已经全过”的自述。
2. 检查实际差异是否超出允许范围，是否覆盖、删除或混入其他会话改动。
3. 必要时重跑当前代码的关键检查，确认关键文件确实进入测试与类型检查。
4. 实际查看页面、截图、HTML、ZIP、PNG 或其他产物，独立核对关键数值。
5. 区分进程结束、测试通过、端到端场景通过和需求验收通过。
6. 区分真实缺陷、采集脚本问题、允许差异与明确未执行事项。
7. mock、替代剪贴板和本地预览不得冒称真实服务或生产验收。

三、输出格式
- 每个必须项分别给出 PASS / FAIL / UNVERIFIED 和证据路径。
- 每个 FAIL 写清：场景、期望、实际、具体数值或截图事实、相关源码位置、
  最小修复方向和需要重跑的复验场景。
- 单独列出未验证、未做和超出授权范围的事项。
- 缺少证据不能判 PASS，允许差异不能误判为缺陷，文件存在不能代替真实可用。

不要修改实现或提前写验收通过记录。把最终结果交回主 Agent，由主 Agent决定交付、定向返工或继续审查。
```

附录 C：主 Agent 自动生成的定向返工 Prompt

> **自动生成说明：**你只需指出“哪里不对、希望怎样”。主 Agent 会把你的反馈和审查证据整理成以下返工任务，不需要你编写 FAIL 编号、复验步骤或技术修复要求。

```
你是本批定向修复执行 Agent。只处理独立审查已经证实的 FAIL，不做新设计，
不扩大业务范围，不替主 Agent 宣布最终验收。

一、输入与边界
- 项目根：<绝对路径>
- 审查结果与失败证据：<路径>
- 只允许修改：<与已证实根因直接相关的文件和新证据目录>
- 保留修复前证据，不覆盖历史截图、日志、下载文件和原始结果。
- 禁止真实模型、支付、部署、发布、外部写入、安装依赖和无关重构。

二、逐项修复
FAIL-<编号>：<问题标题>
- 期望：<用户可见且可测量的正确行为>
- 实际：<当前错误行为、数值和证据位置>
- 已知线索：<相关文件和可能根因，必须先核实，不能直接照抄结论>
- 允许修改：<明确文件或模块>
- 复验：<必须重新执行的真实操作、尺寸、状态、返回值或产物检查>

三、复验要求
1. 先复现或核对失败证据，再做最小修改。
2. 逐项运行原始失败场景，并检查直接相关的旧功能回归。
3. 保存修复前后数据、截图、日志和实际产物，核对内容而不只检查文件生成。
4. 不删除失败测试、不降低阈值、不缩小视口、不强制 DOM 点击、不裁切问题区域。
5. 运行相关专项测试和项目要求的回归检查，记录真实命令与退出码。

四、交回主 Agent
- 每个 FAIL 的根因、修改文件、实测结果、前后证据和未解决项。
- 明确哪些已经通过定向复验，哪些仍未验证或需要继续返工。
- 不勾选最终验收，不归档。主 Agent 复核后再决定是否安排独立终审。
```

附录 D：主 Agent 自动整理的全局 AGENTS.md 示例

自动生成说明：这份规则由主 Agent 根据用户的长期分工偏好整理。用户只需审核内容与文件位置并确认写入，不需要自己编写规则。

```
# 双 Agent 协作默认规则

## 1. 模型与角色协同
- 主 Agent 负责需求、方案、审美与质量审查和最终验收；指挥层可选
  Kimi K3.1 Pro、Codex、Claude，实际以任务表现为准。
- 执行 Agent 负责规则明确、重复耗时的修改、批量处理、测试和资料整理；
  可选 DeepSeek、GLM/ZCode、Codex，先核对宿主、配置和计费。
- 低成本执行可节省 Token，但可能更慢；选择时同时比较质量、总耗时和费用。
- 简单任务直接完成；多文件、长测试或需要独立上下文时委派，只读任务不派实施。
- 执行 Prompt、子智能体提示词和任务书由主 Agent 自动生成；
  用户只负责目标、方案意见、批准和结果反馈。
- 独立审查由未参与实施的只读 Agent 承担，不能用执行者自审代替。

## 2. 默认执行器与启动检查
- 默认执行 CLI 为 zcode；需要 Codex 时使用 codex exec。
- 不假定执行器已安装或已登录。首次使用先核对版本与 --help；
  缺失时如实报告，不自动安装另一套宿主或修改其配置。
- 主、执行双方都固定到同一个明确项目根。新批默认使用独立会话；
  只有确需保留执行上下文时才 resume，不恢复其他项目的会话。

## 3. 授权和范围
- 先读当前项目 AGENTS.md、方案与已有批准，不扩大用户授权。
- 只读核查与本地实施分开；真实模型、支付、外部写入、部署、发布、
  安装依赖、数据库迁移和不可逆操作均需对应明确授权。
- 不默认使用 yolo、dangerously-bypass 或等价的高权限绕过方式。

## 4. 派工任务书
- 每批只设一个清晰目标，写明：项目根、必读输入、前置条件、允许/禁止
  修改范围、实施项、验收命令与场景、证据路径、停止条件和交接格式。
- 复杂任务先保存 TASK.md 再把路径交给执行者；简单任务可传完整提示，
  不为了留文件而打断只读工作。
- 执行提示必须包含“实际执行和验证，不只给计划；不替主 Agent 宣布
  最终验收；如实列出 FAIL、未验证和未做项”。

## 5. 后台与冲突
- 宿主支持后台任务时使用后台执行，记录 task_id，优先等待完成通知；
  不高频轮询，不把进程活着当作有进展。
- 多批实施按依赖串行。只有文件所有权、环境、浏览器空间和产物路径
  均不重叠时才并行；只读资料整理可与实施并行。
- 为每个执行者分配明确可改路径与独立证据目录。禁改他人活跃文件，
  禁止 reset、checkout、整文件覆盖或删除历史证据来消除差异。

## 6. 审查与返工
- 执行完成不等于验收通过。读取完整输出、结果文件、实际差异和证据；
  必要时由主 Agent 重跑关键检查，或派未参与实施的只读 Agent 审查。
- 退出码 0、测试数量、截图数量、文件存在和内部子智能体自述都不能
  单独证明 PASS。检查类型覆盖、真实用户场景和证据身份。
- FAIL 返工只处理已证实问题，写明期望、实际、证据、修改边界和复验。
  不删测试、不降阈值、不缩小场景、不隐藏失败来通过。
- 同一问题两轮仍无法证实修复，或必须越权时，暂停并回主 Agent 复审。

## 7. 最终汇报
- 分别报告：完成内容、静态检查、自动测试、真实操作、证据位置、
  未验证项和遗留风险。明确区分 mock 与真实服务。
- 未经用户确认不归档、不发布；不把 PARTIAL 写成 PASS。
```

11 作者与资料说明

作者：simon，一个不懂代码的AI极客

微信交流群：simonsxxs
GitHub：https://github.com/simonsxxs
Gitee：https://gitee.com/simonsxx
个人网站：https://simonsxx.com

本教程根据主 Agent 与用户分工的实际交互方式整理，只提取“用户怎么说、主 Agent 怎样接续”的协作过程，不展示实际项目名称、任务内容、文件路径、测试结果或验收状态。文中任务编号和示例均为虚构。

参考：Kimi Code 官方文档 Agents and Sub-Agents 与 Built-in Tools。这些资料用于核对主/子 Agent、后台任务和规则文件的基本能力；本教程不要求用户直接使用其中的工具。